

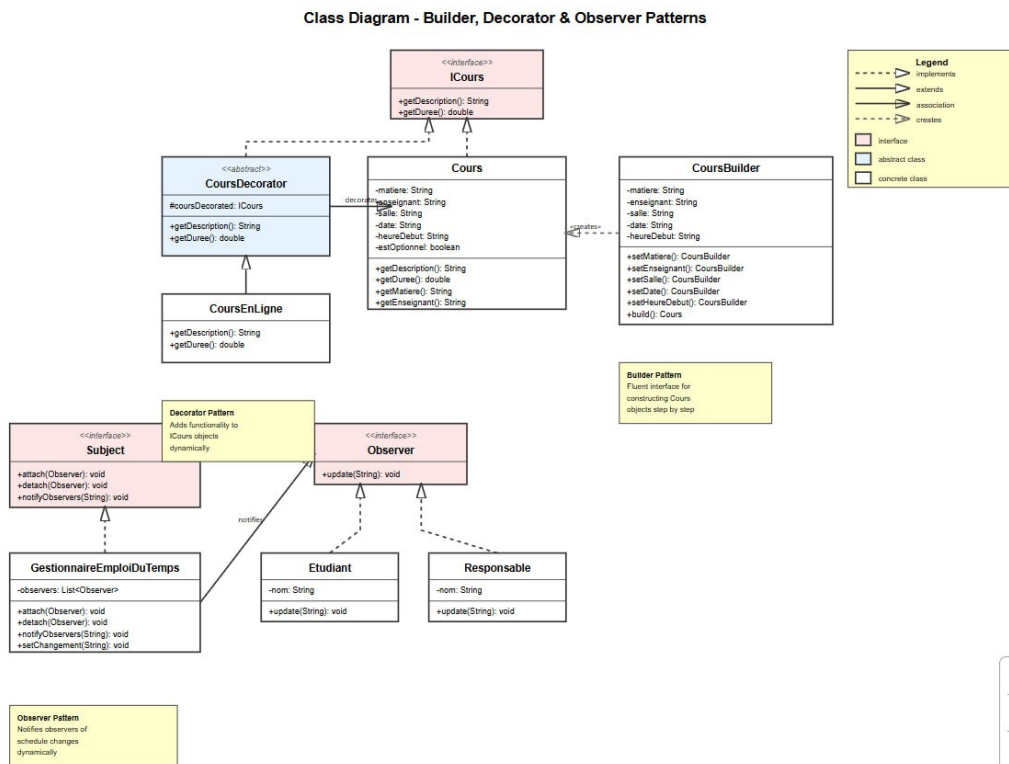
TP Génie Logiciel — Design Patterns

Bonus : Diagramme + Principes SOLID

Nom : **MERZOUGUI Ahmed Reda** Groupe : **SIAD 19**

1 Diagramme de Classe Final

Le diagramme suivant montre l'implémentation des trois design patterns : Builder, Observer et Decorator.



2 Analyse des Principes de Conception Logicielle (SOLID)

Nous avons vérifié si notre code respecte les principes de conception logicielle. Voici l'analyse :

2.1 1. Single Responsibility Principle (SRP)

Chaque classe a une seule responsabilité :

- `CoursBuilder` construit uniquement l'objet `Cours`
- `GestionnaireEmploiDuTemps` gère uniquement les notifications
- `CoursDecorator` ajoute du comportement dynamiquement

Conclusion : SRP bien respecté.

2.2 2. Open/Closed Principle (OCP)

Le Decorator permet d'ajouter de nouvelles fonctionnalités (`CoursEnLigne`, `CoursEnAnglais`) sans modifier la classe `Cours`.

Donc : le code est ouvert à l'extension mais fermé à la modification.

2.3 3. Liskov Substitution Principle (LSP)

Tous les décorateurs implémentent l'interface `ICours` $\Rightarrow$ ils peuvent remplacer un objet `Cours` sans problème.

Conclusion : LSP respecté.

2.4 4. Interface Segregation Principle (ISP)

L'interface `ICours` est simple et ne force PAS l'implémentation de méthodes inutiles.

Risques : si on ajoute trop de méthodes dans `ICours`, ce principe pourrait être violé.

2.5 5. Dependency Inversion Principle (DIP)

Le Subject (`GestionnaireEmploiDuTemps`) dépend de l'interface `Observer` et non des classes concrètes.

Conclusion : DIP bien respecté.

3 Conclusion Générale

- Le code respecte **la majorité des principes SOLID**
- L'architecture est extensible grâce aux Design Patterns
- Le code devient **plus maintenable et modulaire**

Remarque : Une amélioration possible serait d'utiliser une *factory* pour les décorateurs.